1. Лабораторная работа. Множества в языке Паскаль

В языке программирования Паскаль разрешено определять тип объектов-множеств, элементами которых являются значения одного и того же базового типа. Базовый тип определяет перечень всех элементов, которые могу содержаться в данном множестве. Количество элементов, входящих в множество, может меняться в пределах от 0 до 265 (множество, не содержащее элементов, называется пустым).

1.1 Описание множества

Описание типа множества имеет вид:

Type <имя типа> = set of <базовый тип>;

Здесь <имя типа> - идентификатор; <базовый тип> - один из скалярных типов, кроме вещественного. Базовый тип задается диапазоном или перечислением. Из стандартных типов Паскаля в качестве базового типа множества могут быть указаны типы byte, char и boolean. Базовый тип вводится либо через предварительное определение в разделе описаний программы, либо с помощью прямого указания после слов set of в описании типа множества, например:

1. Type letter = ‘a’..’z’; {описание ограниченного типа letter}

Type SL = set of letter; {описание множественного типа SL с базовым типом letter}.

2. Type SL1 = set of ‘a’..’z’; {прямое включение определения базового типа ‘a’..’z’ в описании множественного типа SL1}

Если в программе используются переменные, значениями которых являются множества, то эти переменные описываются обычным образом:

1). Type INTSET = set of byte;
var m1,m2: INTSET; {переменные описаны через указание принадлежности ранее определенному типу};
2). Var m3: set of 1..20; {определение типа переменной непосредственно
включено в ее описание};

Задать значение переменной типа множества в программе можно с помощью оператора присваивания, в правой части которого в квадратных скобках перечислены через запятую элементы множества (так называемый конструктор множества). Примеры значений переменных множественного типа:

1. [ ] – пустое множество;
2. [1,3,5..12] – множество, содержащее элементы 1, 3, 5, 6, …, 12;
3. [‘a’..’p’,’u’,’z’] – множество, состоящее из перечисленных символов типа char.

Элементы типа множества могут задаваться в виде выражений, например: [2+4, 3*2]. Выражения должны иметь значения из данного базисного множества порядкого типа. Область значений переменной множественного типа представляет собой набор всевозможных подмножеств, образованных из элементов базового типа.

В отличие от перечислений нельзя говорить о первом, втором и т.д. элементах множества, поскольку для паскалевских множеств понятие упорядоченности элементов не имеет смысла.

Основные выводы

1. Базовым типом элементов множества может быть только порядковый тип.
2. Число элементов множества не должно превышать 256.
3. Порядковые номера элементов множества (т.е. значения функции ord) должны находиться в пределах от 0 до 255.
4. Переменная множественного типа может получить значение при помощи оператора присваивания (операция считывания значения из входного потока не предусмотрена).
5. Константа типа множества содержит ее элементы, заключенные в квадратные скобки и разделенные запятыми. Каждый элемент такой константы может представлять собой выражение или интервал.

1.2 Операции над множествами

Над переменными множественного типа могут выполняться те же операции, что и над обычными множествами: объединение (+), пересечение (*), разность (-). Кроме того, определены операции проверки принадлежности элемента множеству (in), проверки тождественности множеств (=), нетождественности множеств (<>), определения принадлежности (вложенности) множеств (>= или <=).

Примеры:

	Значение A	Значение B	Выражение	Результат
1.	[1, 2, 4]	[1, 4, 2]	A=B	true
2.	[‘a’..’z’]	[‘a’..’p’]	A=B	false
3.	[1, 2, 5, 6]	[1, 2]	A<>B	true
4.	[‘a’,’b’,’c’]	[‘a’..’z’]	A<=B	true
5.	[‘a’..’k’]	[‘a’..’z’]	A>=B	false
6.	[1, 2, 3]	[1, 4, 5]	A+B	[1, 2, 3, 4, 5]
7.	[1, 2, 3]	[1, 3, 4, 5]	A*B	[1, 3]
8.	[1, 3, 4, 5]	[1, 4, 6]	A-B	[3, 5]

Операция **in** позволяет определить, принадлежит ли элемент множеству или нет. Первым операндом, стоящим слева от слова in, является выражение базового типа. Второй операнд, стоящий справа от слова in, должен иметь множественный тип, например:

a in [a, b, c, d] - результат true
2*4 in [0..4, 7..10] - результат true
3 in [1+2, 4, 5] - результат true

5 in [2, 4, 6, 8] - результат false

При использовании операции in проверяемое на принадлежность значение и множество в квадратных скобках не требует предварительного определения в разделе описаний, если они заданы в виде конкретных значений.

Операция in позволяет проводить эффективно сложные проверки условий. Например, вместо

(c>= ‘0’) and (c<= ‘0’) or (c>= ‘A’) and (c<= ‘Z’)

проще записать

c in [‘0’..’9’, ‘A’..’Z’],

причем последняя конструкция будет, как правило, более эффективной.

Операции (=) и (<>) позволяют проверить, равны ли два множества или нет. С помощью операции (>=) и (<=) можно определить, является ли одно множество подмножеством другого.

Примеры:

[red, white]=[red, green] – результат false;

[1]<=[0..4] - результат true.

Замечание 1. Пустое множество [ ] является подмножеством любого другого множества независимо от базового типа его элементов.

Замечание 2. Множества-операнды могут иметь непересекающиеся базовые типы. Располагая, например, множествами A: set of 1..99 и B: set of 100..150, можно в результате объединения A+B получить новое множество с базовым типом 1..150.

Замечание 3. Следует различать конструктор множества [X..Y] и отрезок порядкого типа X..Y. При X>Y в первом случае речь идет о пустом множестве, а во втором компилятор Паскаля зафиксирует ошибку.

При проверке на подмножество выполняется тест на «меньше или равно», а не только проверка на собственное подмножество, т.е. без «равно». Операции (<) и (>) в Паскале не предусмотрены, поэтому при необходимости проверку на собственное подмножество для множеств A и B можно провести следующим образом:

(A<=B) and (A<>B) или (A>=B) and (A<>B)

Для задания правильного выполнения порядка операций следует учитывать принятый в Паскале порядок старшинства (приоритета) операций над множествами: пересечение (*) имеет тот же приоритет, что и арифметические операции умножения и деления; объединение(+) и разность(-) занимают следующий, более низкий уровень приоритета, аналогично арифметическим операциям сложения и вычитания; на самом нижнем уровне находятся операции сравнения множеств (=, <>, <=, >=) и проверки принадлежности элемента множеству (in). Операции одного приоритета выполняются слева направо. Для изменения порядка выполнения операций используются круглые скобки.

Правила для операций с множествами можно записать в форме:

Уровень приоритета	Операция	Тип операндов	Результат
1	*	множество	множество
2	+ -	множество множество	множество множество
3	= <> >= <= In	множество множество множество множество слева: выражение, справа: множество	boolean boolean boolean boolean boolean

1.3 Использование множеств

Наиболее эффективно множество может быть использовано для замены операторов if, например, для проверки наличия некоторого ответа в списке разрешенных.

Приведенная ниже программа ввода строки символов, содержащей латинские буквы, цифры, пробелы с контролем правильности введенных символов, может служить примером использования множеств.

```
program EX;
    var
        Str:string;
        L:byte;
        t: boolean;
begin
        writeln (‘Введите строку’);
         readln (Str);
         L:=length (Str);    {число введенных символов}
         t:=L>0;             {true, если не пустая строка}
    while t and (L>0) do
         begin
             T:=Str[L] in [‘0’..’9’, ‘A’..’Z’,’a’..’z’];
                          {проверка допустимости символа}
             dec (L)          {предыдущий символ}
         end;
```

```
        if t then writeln ('Правильная строка')
            else writeln ('Неправильная строка')
    end.
```

1.4 Вопросы и упражнения

1. Сколько различных значений может принимать переменная AS, имеющая описание var AS: set of 0..2?
2. Эквивалентны ли выражения p in [1, 2, 5] и (p=2) or (p=2) or (p=5)?
3. С использованием операции in организовать проверку текущего числа цифр в строке s, описанных как var s: string [62].
4. Вычислить:

a) [1, 4, 7] + [2, 3] + [5, 6];
b) [1, 4, 7] * [3, 5];
c) [1, 4, 7] – [2, 5];
d) [2..10] – [4, 6] – [2..12] * [8..15].

5. Упростить (A, B - множества):

a) A * B – A;
b) A – (A – B);
c) (A + B) – (A – B) – (B – A).

6. Определить значения:

a) [2]=[2, 2, 2];
b) [1, 3] <> [3, 1];
c) [1, 2, 3] = [1..3];
d) ['a', 'b'] = ['b'..'a'].

7. Дан текст из строчных латинских букв. Составить программу определения количества гласных букв (a, e, i, o, u), встречающихся в тексте.

1.5 ЗАДАНИЕ

1. Организовать процедуру для задания элементов множества A. Организовать процедуру printset, печатающую элементы множества A с определением их числа.
2. Описать множество B. Ввести значения его элементов. Распечатать множество B с помощью процедуры printset.
3. Задать два текстовых предложения. Сравнить при помощи множеств эти предложения, не используя стандартные операция для работы со множествами (такие как + - * /) и найти общие слова, встречающиеся в обоих предложениях.
4. Задать множество S состоящее из символов. Разделить множество S на три подмножества: A – символы русского алфавита, Б – латинские символы, B – цифры.

2. Лабораторная работа. Работа с файлами в ПАСКАЛЕ

Традиционно под файлом понимается именованная совокупность данных на внешнем носителе, однако, в Паскале этот термин трактуется более широко. Здесь файлом считается также любое внешнее устройство, по своему назначению являющееся источником или приемником информации, например, дисплей, клавиатура и т.д. Такое устройство называется логическим, если учитывается только его основная функция, а не физические характеристики. Для того чтобы указать, откуда считываются данные и куда данные должны записываться, в Паскале и вводится понятие файла (file).

Файлы в Паскале представляют собой структуру данных одного типа. Число компонентов файла при описании файла жестко не фиксируется.

Программирование на уровне абстракции, называемой файлом, сводится к тому, что до начала операций ввода, вывода конкретному внешнему файлу должна быть поставлена в соответствие переменная файлового типа. После этого необходимо открыть файл для чтения, записи или чтения и записи информации совместно. Только после этих действий может быть выполнена требуемая операция.

Описание и обработка файлов

Файловый тип в Паскале имеет три разновидности: различают типизированные, нетипизированные и текстовые файлы. Объявление первых двух видов файлов может быть дано в следующей форме:

Type <имя файловой переменной>= file [of <тип>]

Здесь <тип> - тип каждой из записей типизированного файла. Если зарезервированное слово of и параметр <тип> опущены, объявляемый файл является нетипизированным. При этом под записями подразумеваются компоненты файла, которые могут иметь любой, но один и тот же тип, за исключением файлового или любого структурированного типа, содержащего элементы типа файл. Каждая запись файла имеет номер, причем считается, что первой записи присвоен нулевой номер.

Замечание. Не следует смешивать записи файлов с данными комбинированных типов (record), которые также обычно называют записями.

Как типизированные, так и нетипизированные файлы в основном используются в режиме последовательного доступа, когда ввод и вывод записей ведутся последовательно, в порядке возрастания их номеров. Следует учитывать, что с этими файлами можно работать и в режиме произвольного доступа, когда возможно выборочное обращение к конкретным записям, которые задаются их номерами.

В Паскале определен также стандартный файловый тип, обозначаемый при объявлении служебным словом text:

type <имя>= text

Текстовый файл представляет собой последовательность строк переменной длины. Каждая строка завершается признаком конца строки EOLN. Текстовые файлы могут использоваться как для ввода, так и для вывода информации, но только в режиме последовательного доступа.

Примеры объявлений файловых типов и переменных:

```
type a=file of char;
       b=file of integer;
var tf:text;
    lt: a;
    u: file;
    ff, fs: b;
```

В приведенных описаниях определены типы символьного файла a и целочисленного файла b, файловые переменные tf текстового типа; lt типа a, ff и fs типа b, также файловая переменная u без типа.

Существует возможность определения типа файла и непосредственно при введении файловых переменных:

var f1,f2: file of integer.

Файловые переменные имеют специфическое применение. Над ними нельзя выполнять никаких операций. Можно использовать их лишь для операций с файлами (чтение, записи, удаление и т.д.). К роме того, через файловую переменную можно получить информацию и конкретном файле (имя, тип и т.д.).

Для работы с файлами всех трех типов существуют общие правила:

1. В Паскале файл является некоторой переменной, следовательно, ему следует присвоить имя. С другой стороны операционная система использует имена файлов на различных носителях. Между именами файлов в Паскале и именами файлов, присваиваемых операционный системой, нужно установить связь. Для этого имеется стандартная процедура assign? Вызов которой осуществляется следующим образом

Assign(<имя>, <файл>),

Здесь <имя> - имя файловой переменной,<файл> - строковое выражение, задающее имя логического устройства или файла. Последний может существовать на диске либо создаваться заново.

Пример:

```
Var f : file of integer;
…
Assign(f , 'c:\mk\data.dat').
```

2. Файл f следует открыть для выполнения планируемых операций с помощью одной из трех процедур: Reset(f), Rewrite(f) или Append(f), входящих в стандартный модуль System. Существующий файл открывается для считывания с помощью процедуры Reset, а для создания нового файла и подготовки его к записи применяется процедура Rewrite.

Типизированные нетекстовые файлы всегда открываются с помощью процедур Rewrite и Reset, причем последняя делает файл доступным не только для считывания, но и для записи или дозаписи.

Для дописывания текстового файла необходимо его открыть с использованием процедуры Append(f).

Замечание. С помощью процедуры Rewrite открывается новый пустой файл и ему приписывается имя, заданное процедурой Assign. Если файл с таким именем уже существует, то он уничтожается.

3. С помощью стандартных процедур Read(f , <список вывода>), Readln(f , <список вывода>) или Write(f , <список вывода>), Writeln(f , <список вывода>) можно соответственно считать данные из файла f или записать данные в файл. В этих операторах f – файловая переменная; <список ввода-вывода> - это перечисленные через запятую имена переменных при вводе и перечисленные через запятую выражения, значения которых выводятся при выводе. Процедуры Readln, Writeln предназначены для работы со строками и могут использоваться без указания списка ввода – вывода.

Замечание. Типизированные Файлы строятся из записей фиксированного размера, который задается неявным образом при объявлении типа файла. Поэтому такие файлы, открытые процедурой Reset, могут быть модифицированы путем переменного обращения к процедурам Read и Write.

Фиксированный размер записей типизированных файлов позволяет организовать прямой доступ к записям. Для этого используется стандартная процедура Seek, обращение к которой имеет вид:

Seek(f , <номер>)

Аргумент <номер> - это выражение типа longint, задающее номер записи.

4. После завершения работы с файлом с помощью стандартной процедуры Close(f) файл закрывается. При этом дописывается метка конца файла EOF. Закрытие файла с помощью процедуры Close позволяет обеспечить сохранность информации.

Замечание. При попытке закрыть с помощью процедуры Close файл, который не был до этого открыт или который уже закрыт, возникает ошибочная ситуация.

Процедуры rename, erase; функция eof

1. Процедура Rename позволяет переименовывать существующий файл, связанный с указанной в качестве параметра файловой переменной.

Пример.

```
Assign(f , ‘c:\test.txt’);
Rename(f , ‘c:\test.old’).
```

В результате работы этого фрагмента программы файл c:\test.txt, связанный с файловой переменной f, будет переименован в файл c:\test.old.

Замечание. Процедура Rename может быть применена только к закрытым файлам.

2. Процедура Erase стирает существующий файл с диска. При этом файловая переменная должна быть предварительно связана с реально существующим файлом. Например, файл test.old, если он действительно существует, может быть удален с диска с помощью следующих двух операций:

```
Assign(f , ’c:\test.old’);
Erase(f).
```

3. Используя логическую функцию EOF (End of File – конец файла), в процессе считывания информации можно проверить, достигнут ли конец файла, т.е. находится ли указатель файла за последним элементом или нет. Если f – файловая переменная, то EOF(f) возвращает true, когда конец файла достигнут.

Замечание. При применении процедуры Append открывается файл и указатель сразу устанавливается в конец файла.

Подпрограммы для нетипизированных файлов

Рассматриваемые файлы совместимы с любыми файлами, кроме текстовых, независимо от их типов и структуры. Для выполнения операций обмена с файлами без типа в Паскале предусмотрены специальные процедуры BlockRead и BlockWrite, заменяющие обычные Read и Write. При этом появляются возможности для быстрого копирования информации.

Процедуры BlockRead и BlockWrite осуществляют передачу информации не записями, а блоками (их размер по умолчанию предполагается равным 128 байтам, но может быть изменен при указании значения второго (необязательного) параметра в процедурах Reset, Rewrite).

Для размещения блока необходимо предусмотреть выделение буфера в виде переменной, занимающей в памяти область соответствующего размера (удобно использовать массив с элементами длиной в байт).

Формат обращения к процедурам BlockRead и BlockWrite:

```
BlockRead(f , <буфер>, <количество записей в блоке>);
BlockWrite(f , <буфер>, <количество записей в блоке>);
```

Здесь <буфер> - имя переменной, используемой в качестве буфера.

Пример (программа копирования):

```
Var inf , outf : file;
        Buffer : array [1..512] of byte;
Begin
        Assign(inf , ‘a:\fst’); {файл-источник}
        Assign(outf , ‘c:\sec’); {файл-приемник}
        Reset(inf , 512);
```

```
        Rewrite(outf , 512);
                While not EOF (inf) do
                Begin
                        BlockRead(inf,buffer,1); {копирование данных}
                        BlockWrite(outf,buffer,1); {из источника в приемник}
                End;
        Close(outf);
        Erase(inf); {удаление файла-источника}
End.
```

Основные выводы

1. Файл в Паскале – это абстрактная модель физического набора данных, находящихся обычно вне программы.
2. Число компонентов файла при описании не фиксируется.
3. Различают типизированные, нетипизированные и текстовые файлы.
4. Конкретный внешний файл должен быть связан с файловой переменной (переменной файлового типа) с помощью процедуры Assign.
5. Для выполнения операций чтения или записи файл следует открыть с помощью одной из трех процедур: ReWrite, Reset или Append (последняя используется для дозаписи текстового файла).
6. Операции считывания или записи в файл выполняются с помощью стандартных процедур Read, Write.
7. Завершение работы с файлом требует его закрытия с помощью процедуры Close.
8. Файлы в основном используются в режиме последовательного доступа, однако для нетекстовых файлов возможно обращение к конкретным записям, которые задаются их номерами.
9. Для быстрого копирования информации можно использовать нетипизированные файлы, для которых передача информации производится не записями, а блоками.

Задание к лабораторной работе

1. Составить и отладить программу определения количества отрицательных элементов в файле действительных чисел:
   - 1.1 из программы создать файл на диске, задав ему произвольное имя;
   - 1.2 заполнить файл с произвольными положительными и отрицательными числами;
   - 1.3 открыть файл для чтения и определить количество отрицательных чисел.
2. Разработать и отладить процедуры, добавляющие заданную букву в произвольное место символьного файла.
3. Создать на диске Текстовый файл, ввести в него произвольный текст, затем вывести n строку текста, предусмотреть возможность отсутствия строки с таким номером.
4. Реализовать функцию поиска в Текстовом файле подстроки. Результатом поиска является пара значений номер сроки – номер позиции. Предусмотреть возможность более одного совпадения.
5. Задать текстовый файл. Написать функцию определяющую количество уникальных слов в файле. Написать функцию определяющую количество повторов каждого слова, содержащегося в файле.

Примечания

Запрещается загружать файлы целиком в оперативную память.

3. Лабораторная работа. Работа с указателями и ссылочными переменными

Адреса в персональных компьютерах состоят из двух шестнадцатеричных слов – адреса сегмента и смещения. Сегмент может начинаться только с физического адреса кратного 16. Смещение показывает положение участка памяти относительно начала сегмента и представляет собой число, определяющее номер байта, к которому производится обращение. Задав нужное смещение, можно обратиться к любому байту данного сегмента.

Указатели в Турбо Паскале

Для работы с динамической памятью в Турбо Паскале используются указатели. *Указатель — это переменная, значением которой является адрес л оперативной памяти.* Указатель занимает четыре байта и содержит в двух старших байтах адрес сегмента, а в двух младших – смещение.

С помощью указателей можно размещать в динамической памяти переменные любых типов. Указатель в этом случае будет ссылаться па первый байт области памяти, выделенной под переменную. Указателя могут связываться с некоторым типом данных (типизированные *указатели)* или не связываться *(нетипизированные указатели).*

Объявление соответствующих указателей в программе производится следующим образом:

1. Для типизированных указателей надо ввести имя типа данных, с которыми будут связаны указатели. Другими словами, после знака ^, с помощью которого объявляется указатель, должен следовать идентификатор, например:

```
TYPE DI = 1..15;
IPTR = ^Integer; DPTR = ^DI;
var PINT: IPTR; PD: DPTR;
 PH:  ^ integer ;
```

2. Для нетипизированных указателей используется стандартный тип данных –POINTER, например:

```
Vat pp: POINTER;
```

1.2. Операции с указателями:

Для указателей в Турбо Паскале допустимы только операции присваивания и сравнения. Указателю можно присваивать содержимое другого указателя *того же типа* или стандартную константу NIL (пустой указатель), совместимую по присваиванию с указателями любых типов.

Замечание. Нетипизированные указатели совместимы по присваиванию с указателями любых типов. Поэтому с их помощью можно преодолеть запрет на присваивание между указателями, связанными с разными типами данных.

Доступ к значению, хранящемуся в памяти по адресу, на который ссылается указатель, может быть получен, если после идентификатора указателя поставить символ ^ (карет). Таким образом в программе производится обращение к динамическим переменным, например:

```
PINT^ :=4;
```

При этом в ячейку памяти, на которую указывает указатель PINT, будет записано число 4.

Задание к лабораторной работе

Реализовать программу эмулирующую работу с динамической памятью.

1. Определить структуры данных представляющие собой:1 статический сегмент памяти и 4 сегмента динамической памяти. Размер каждого сегмента 32 байта. В качестве сегментов памяти можно использовать массивы.
2. Реализовать функцию размещения в статическом сегменте указателя имеющего один из трех типов byte (1 байт), int (2 байта), longint (4 байта), и выделения памяти для хранения значения, на которое ссылается указатель. Например, NewPointer('p', byte). Сегмент для размещения должен выбираться автоматически.
3. Реализовать функции чтения и присвоения значений на которые ссылаются указатели. Например WritePointer(‘p’, 5), ReadPointer(‘p’).
4. Реализовать функцию присвоения значения одного указателя другому. Например SetPointer(‘p’,’b’).
5. Реализовать функции освобождения памяти. Например, FreePointer(‘p’).

Примечания

Предусмотреть невозможность выделения памяти в случае нехватки места в сегментах. При чтении указателя обрабатывать возможность пустого значения. Резервировать динамическую память при выделении памяти для указателя. При выделении памяти под уже существующий указатель эмулировать процесс утечки памяти. Запрещается хранить информацию о указателях вне статического сегмента. Статический сегмент должен содержать информацию о имени указателя, номере сегмента, байте смещения, размере или типе указателя.

4. *Лабораторная работа*. Методы внутренней сортировки массивов

***Цель работы* — изучение эффективности основных методов внутренней сортировки массивов.**

Упорядочение данных (сортировка) процесс, используемый в прикладном программировании очень часто. При больших объёмах данных именно методы сортировки определяют эффективность всей системы. Цель сортировки — облегчить поиск элементов в отсортированном множестве данных.

1.1. Постановка задачи

В общем виде задача сортировки выглядит следующим образом. Пусть надо упорядочить n элементов R_1, R_2, …, R_n, которые являются записями. Каждая запись R_j имеет свой ключ K_j, который и управляет процессом сортировки. Помимо ключа, запись может содержать дополнительную информацию, которая не влияет на сортировку, но присутствует в записи.

Задача сортировки — найти такую перестановку записей

$$R_{p1}, R_{p2}, \ldots, R_{pn},$$

чтобы ключи расположились бы в следующем порядке K_{p1}, K_{p2},…, K_{pn}.

При сортировке либо перемещаются сами записи, либо создаётся вспомогательная таблица, которая описывает перестановку и обеспечивает доступ к записям в соответствии с порядком их ключей.

В дальнейшем под множеством данных будем понимать массив A[n] целых или вещественных чисел. Задача сортировки такого массива состоит в преобразовании массива $a[1], a[2], \ldots, a[n]$ в массив $a[k_1], a[k_2], \ldots, a[k_n]$. Обычно сортировку подразделяют на два класса: внутреннюю, когда все записи хранятся в оперативной памяти, и внешнюю, когда при сортировке используются внешние запоминающие устройства. Далее рассматривается только внутренняя сортировка.

Основные требования к методам сортировки:

1. **Экономия памяти.** Все перестановки элементов массива должны выполняться на том же месте (то есть должен быть отсортирован исходный массив A, а не получен результирующий массив B).
2. **Устойчивость сортировки.** Все элементы с равными значениями после сортировки должны сохранять порядок следования относительно друг друга.

Для оценки быстродействия алгоритмов различных методов сортировки обычно используют два показателя:
– количество присваиваний;
– количество сравнений.

Все методы внутренней сортировки можно разделить на две группы:
– прямые методы сортировки;
– улучшенные методы сортировки.

Прямые методы сортировки в соответствии с принципами, положенными в основу методов, разделяются на три подгруппы:

1. Сортировка **прямым включением**;
2. Сортировка **прямым выбором**;
3. Сортировка **прямым обменом** (метод “пузырька”).

Улучшенные методы сортировки основываются на тех же принципах, что и прямые, но используют некоторые оригинальные идеи для ускорения процесса сортировки. Прямые методы на практике используются довольно редко, так как имеют относительно низкое быстродействие. Однако они хорошо показывают суть основанных на них улучшенных методов. Кроме того, в некоторых случаях (как правило, при небольшой длине массива или особом исходном расположении элементов) некоторые из прямых методов могут даже превзойти улучшенные методы.

В данной лабораторной работе необходимо изучить ряд известных алгоритмов сортировки и создать программу, реализующую

1. Метод сортировки прямым включением;
2. Метод сортировки прямым выбором;
3. Метод сортировки прямым обменом;
4. Метод быстрой сортировки.

Разрабатываемая программа должна обеспечивать

- вывод на экран меню;
- ввод исходной информации;
- формирование массивов с большим числом элементов;
- выбор метода сортировки;
- сортировку массива;
- печать результата;
- замеры времени выполнения сортировки массива;
- подсчёт числа сравнений и перемещений элементов в процессе сортировки.

Демонстрация работоспособности разработанной программы должна обеспечивать два варианта контроля: контроль работоспособности каждого из методов и контроль временных характеристик всех реализованных методов.

1.2. Учебно-методические цели работы

Данная задача представляет собой вариант широко распространенной задачи сортировки, которая применяется в широком классе реальных задач, и умение и знание алгоритмов сортировки и их особенностей является необходимой частью образования специалиста по программному обеспечению. Наряду с
ознакомлением с проблемой упорядочивания массивов и методами ее решения, выполнение работы направлено на достижение следующих учебно-методических целей:

• приобретение навыков и приемов использования массивов, доступа к их элементам и преобразований массивов;
• практическое освоение принципов программирования ;
• освоение и изучение встроенных функций (применение генератора псевдослучайных чисел и замеры датчиков времени);
• получения навыков сравнительного анализа эффективности разных алгоритмов.

1.3. Рекомендации по выполнению работы

Программа должна обеспечить сортировку массивов размером произвольной длины до 10000 элементов и выводить для контроля:
- при небольшом количестве элементов (например, менее 20) - неупорядоченный массив и массив после сортировки для каждого из предложенных алгоритмов;
- при значительном объеме данных выводить число сравнений и обменов элементов и время сортировки одного и того же массива для всех четырех предложенных алгоритмов.

Содержимое массива рекомендуется формировать с помощью генератора
псевдослучайных чисел.

В ходе выполнения лабораторной работы предполагается реализация методов для четырех широко используемых *алгоритмов сортировки*:

• метод прямого выбора;
• метод прямого включения;
• метод прямого выбора;
• метод быстрой сортировки Хоара.

Для простоты изложения описания алгоритмов проводятся на примере задач сортировки массивов по возрастанию.

1.4. Сортировка с помощью прямого включения

Принцип метода:

Массив длиной N разбивается на две части: упорядоченную и неупорядоченную. Элементы из неотсортированной части поочерёдно выбираются и вставляются в отсортированную часть так, чтобы не нарушать в ней упорядоченность элементов. В начале работы алгоритма в качестве отсортированной части массива принимают только один первый элемент, а в качестве неотсортированной части — все остальные элементы.

Алгоритм состоит из **n-1** прохода (**n** – размер массива), включающего следующие действия:

- взятие очередного **i**-го неотсортированного элемента и сохранение его во вспомогательной переменной **x**;
- поочерёдный сдвиг элементов отсортированной части массива вправо (начиная с **i-1** элемента) с целью определения позиции **j**, в которой присутствие взятого элемента **x** не нарушит упорядоченности элементов;
- включение взятого элемента x в найденную **j**-ю позицию.

Алгоритм метода прямого включения можно представить в виде:

для i=2 до n выполнить {взятие очередного элемента}
$x = a[i];\ j = i - 1;$
пока ($(x < a[j])$ **И** $(j >= 1)$**)** {поиск места включения}
$a[j+1] = a[j];$
$j = j - 1;$
всё-пока
$a[j+1] = x;$ {вставка элемента}
всё-для

Схема работы алгоритма прямого включения на примере массива 3,7,2,14,5,1,8 может быть представлена следующим образом:

Итерация	Результат итерации
1	3 7 | 2 14 5 1 8

2	2 3 7 | 14 5 1 8
3	2 3 7 14 | 5 1 8
4	2 3 5 7 14 | 1 8
5	1 2 3 5 7 14 | 8
6	1 2 3 5 7 8 14 |

1.5. Сортировка с помощью прямого выбора

Аналогично, как и в методе прямого включения, исходный массив разбивается на две части: отсортированную и неотсортированную.

Принцип метода:

- выбирается наименьший элемент массива;
- наименьший элемент обменивается местами с первым элементом;
- процесс повторяется с оставшимися n-1, n-2, … элементами до тех пор, пока не останется один, самый большой элемент.

Схема процесса сортировки с помощью прямого выбора на примере массива 3,7,2,14,5,1,8 выглядит следующим образом:

Итерация	Результат итерации
1	1 | 7 2 14 5 3 8
2	1 2 | 7 14 5 3 8
3	1 2 3 | 14 5 7 8
4	1 2 3 5 | 14 7 8
5	1 2 3 5 7 | 14 8
6	1 2 3 5 7 8 | 14

Метод прямого выбора в некотором смысле противоположен методу прямого включения. Если при прямом включении для внесения одного элемента в упорядоченную часть массива требуется просмотреть *все элементы упорядоченной части*, то при прямом выборе для внесения элемента надо просматривать *все элементы неупорядоченной части*.

Алгоритм метода прямого выбора можно представить в виде:

для i=1 до n-1 выполнить {определение места для размещения очередного элемента}

$x = a[i]; k = i;$

для $j = i+1$ **до** n **выполнить** {поиск наименьшего в неотсортированной части}

если $a[j] < x$ **то** $k = j; x = a[j]$ **всё-если**

всё-для

$a[k] = a[i];$ {обмен местами}

$a[i] = x;$

всё-для

1.6. Сортировка с помощью прямого обмена

Принцип метода:

Справа налево поочерёдно сравниваются два соседних элемента, и если их взаиморасположение не соответствует заданному условию упорядоченности, то они меняются местами.

- После завершения первой итерации наименьший элемент массива будет помещён в самый левый край массива – в элемент с индексом 1.
- На второй итерации выполняются те же действия, но процесс завершается перед элементом $a[1]$, который уже помещён на своё место. Вторая итерация приводит к размещению в позиции $a[2]$ второго по величине элемента заданного массива.
- Таким образом, каждая итерация приводит наименьший элемент в неотсортированной части массива в самую левую позицию этой части, после чего отсортированная часть массива за счёт этого элемента по количеству возрастает на единицу.

Этот метод широко известен как метод «пузырька» (BubbleSort), так как при каждой итерации наименьшие элементы «всплывают» к левому краю массива.

Схема процесса «пузырьковой» сортировки для массива 3,7,2,14,5,1,8 может быть представлена в виде:

Итерация	Результат итерации
0	3 7 2 14 5 1 8
1	1 | 3 7 2 14 5 8
2	1 2 | 3 7 5 14 8

3 1 2 3 | 5 7 8 14
4 1 2 3 5 | 7 8 14
5 1 2 3 5 7 | 8 14
6 1 2 3 5 7 8 | 14

Замечание.

1. При прямом обмене *тяжёлые* элементы опускаются на «дно» медленно (на один уровень при каждой итерации), а *лёгкие* всплывают на «поверхность» очень быстро.
2. В рассмотренном примере последние четыре прохода не повлияли на взаимное расположение элементов массива, то есть фактически сортировка завершилась раньше, чем выполнен алгоритм сортировки. С учётом этого можно предложить вариант улучшения алгоритма – запоминать, были или не были перестановки в процессе прохода.

Алгоритм метода прямого обмена можно представить в виде:

для i=2 **до** n **выполнить**
для j =n **до** i **шаг** (-1) **выполнить**
если $a[j-1] > a[j]$ **то**
$x = a[j-1];$
$a[j-1] = a[j];$
$a[j] = x;$
всё-если
всё-для
всё-для

1.7. Метод быстрой сортировки (метод Хоара)

Метод «пузырька» является наименее эффективным из прямых методов сортировки, так как требует большого количества сравнений и обменов. Однако быстрая сортировка, являющаяся усовершенствованием метода прямого обмена, представляет собой самый хороший из известных методов. Быстрая сортировка, предложенная Хоаром, использует обмены на больших расстояниях.

Метод Хоара основан на следующем алгоритме:

1. Выбирается какой-либо элемент x массива A (например, вблизи середины массива).
2. Элементы исходного массива перемещаются так, чтобы меньшие или равные x были расположены левее x, а большие или равные x — правее x. После такого разделения массива x будет находиться на своём месте в отсортированном массиве, то есть один элемент будет «отсортирован».
3. Процесс разделения применяется к получившимся частям массива (меньше и больше x), затем к частям частей до тех пор, пока каждая из частей не будет состоять из одного и только одного элемента.

Разделение массива можно проводить в соответствии со следующим алгоритмом:

$i = 1;\ j = n;$
$x = a[(\textit{индекс вблизи середины массива})]$
повторять {цикл с постусловием}
пока $(a[i] < x)$ **выполнить** $i = i + 1$ **всё-пока**
пока $(a[j] > x)$ **выполнить** $j = j - 1$ **всё-пока**
если $(i \le j)$ **то**
$w = a[i];\ a[i] = a[j];\ a[j] = w;$
$i = i + 1;\ j = j - 1;$
всё-если
до $(i \le j)$ {окончание цикла с постусловием}

Вариант процедуры быстрой сортировки на Паскале:

```
procedure QuickSort;
    procedure sort(L, R: integer);
    Var i, j: integer; x :< тип элементов >;
    BEGIN i := L ; j := R;
      x := a[(L + R) div 2];
      repeat
```

```
        while  a[i] < x do  i := i + 1 end;
        while  x < a[j] do  j := j - 1 end;
        if i ≤ j then begin
        w := a[i]; a[i] := a[j]; a[j] := w; i := i + 1; j := j - 1;
                end;
    until i > j;
     if L < j then sort(L, j);
     if i < R then sort(i, R);
  END;
BEGIN sort(1, n)
END;
```

Замечание.

1. Процедура Sort рекурсивна. Для размещения в стеке переменных и значений параметров при рекурсивном обращении необходима дополнительная память и время.
2. Процедура QuickSort эффективна лишь для достаточно больших массивов, так как на организацию рекурсии необходимо время.

1.8. ЗАДАНИЕ

1. Сравнить эффективность прямых методов сортировки (число итераций, сравнений и обменов) для числовых массивов, содержащих различное число элементов (20, 500, 1000, 3000, 5000, 10000), выбираемых случайным образом. Для 20 элементов предусмотреть ввод с клавиатуры. Оценить время сортировки, построить соответствующие таблицы.
2. Исследовать влияние начальной упорядоченности массива (уже отсортированный, отсортированный в обратном порядке, частично отсортированный – при разных размерах отсортированной части 25%, 50% и 75%).
3. Сравнить эффективность быстрой сортировки и прямых методов. Определить размеры массивов, когда прямые методы эффективнее. Составить таблицы, иллюстрирующие сделанные выводы.
4. В отчёте привести тексты программ с необходимыми комментариями.

ЛИТЕРАТУРА

1. Кнут Д. Искусство программирования для ЭВМ в 3 т.- М.: Мир, 1978.- Т.3. Сортировка и поиск.
2. Вирт Н. Алгоритмы и структуры данных. – С.-Пб.: Невский диалект.

5. Лабораторная работа. Линейный и бинарный поиск

***Цель работы* — изучение эффективности основных методов поиска в массивах.**

1. АЛГОРИТМЫ ПОИСКА

Алгоритмы поиска, наряду с алгоритмами сортировки, являются основными алгоритмами обработки данных как системных, так и прикладных задач.

Задача поиска состоит в отыскании записи R, имеющей ключ K. Этот ключ называется аргументом поиска.

Алгоритмы поиска применяются для нахождения, например, в массиве элемента с нужными свойствами. Если множество данных задано в виде массива A=$(a[1], a[2], \ldots, a[n])$ из n целых или вещественных чисел, то поиск состоит в нахождении числа $i(1 \le i \le n)$, для которого $a[i] = x$, если x –ключ поиска. Обычно различают постановки задачи поиска для первого и последнего вхождения элемента.

1.1 ЛИНЕЙНЫЙ ПОИСК

Для массива, данные в котором не упорядочены сортировкой или другим способом, единственный путь для поиска заданного элемента состоит в сравнении каждого элемента массива с заданным. При совпадении некоторого элемента массива с ключом, его позиция в массиве фиксируется, и поиск заканчивается успешно. В противном случае будут просмотрены все элементы, а поиск окажется безуспешным. Этот подход называется последовательным или линейным поиском.

Алгоритм линейного поиска имеет вид:

$i = 1;$

пока ($(i \le n)$ **И** $(a[i] \ne x)$**) выполнить**

$i = i + 1;$

всё-пока

Равенство $i = n + 1$ означает отсутствие в массиве A ключа поиска.

Эффективность этого метода очень низкая, так как для отыскания одного элемента в массиве размера N в среднем нужно сделать N/2 сравнений.

Линейный поиск осуществляется циклом (**while** или **repeat** - **until**) с двойным условием. Первое условие контролирует индекс на принадлежность массиву, например, (i<=N). Второе условие - это условие поиска. В цикле **while** это условие продолжения поиска: (A[i]<>X), а в цикле **repeat** - **until** это условие завершения поиска: (A[i]=X). В теле цикла обычно пишется только один оператор: изменение индекса в массиве.

После выхода из цикла необходимо проверить, по какому из условий мы вышли. В операторе **if** обычно повторяют первое условие цикла. Можно говорить об успешном поиске с циклом **while** при выполнении этого условия, а с циклом **repeat** - **until** при его нарушении.

***ПРИМЕР**: Линейный поиск*

```
program PoiskLin;
    var  A: array [1..100] of integer;
      N, X, i : integer;
  begin
     read (N);                                  {N<=100}
     for i :=1 to N do read(A[i]);
     read (X);
     i:=1;                                  {i:=0; для цикла с постусловием}
   while (i<=N) and (A[i]<>X) do i:=i+1;
  {repeat i:=i+1; until (i>N) or (A[i]=X); для цикла с постусловием}
     if i<=N then write('первое вхождение числа ',X,'
                          в массив A на ',i,' месте')
             else write('не нашли');
  end.
```

При поиске последнего вхождения после ввода должны идти операторы:

```
i:=N;                                       {i:=N+1;}
while (i>=1) and (A[i]<>X) do  i:=i-1;
{repeat i:=i-1; until (i<1) or (A[i]=X);}
if i>=1 then write('последнее вхождение числа ', X ,' в массив A на ', i,     'позиции ')
else  write ('не нашли');
```

1.2 ПОИСК С БАРЬЕРОМ

Единственно возможное убыстрение алгоритма линейного поиска заключается в упрощении логического условия. Для этого следует увеличить количество элементов массива A на один элемент со значением ключа поиска x, то есть $a[n+1] = x$. Этот дополнительный элемент называется «барьером». Его введение позволяет алгоритм линейного поиска представить в виде:

$i = 1;$

пока $(a[i] \ne x)$ **выполнить**

$i = i + 1$

всё-пока

Введение барьера позволяет не проверять каждый раз в цикле условие, связанное с границами массива.

Выход из цикла, в котором теперь остается только условие совпадения с ключом поиска, может произойти либо на найденном элементе, либо на барьере. Таким образом, после выхода из цикла проверяется, не барьер ли мы нашли? Вычислительная сложность поиска с барьером меньше, чем у линейного поиска, но также является величиной того же порядка, что и N - количество элементов массива.

Существует два способа установки барьера: дополнительным элементом или вместо крайнего элемента массива.

***ПРИМЕР**: Поиск с барьером*

```
program Poisk_Bar1;
        var      A:array[1..101] of integer;
             N,X,i:integer;
begin
       read(N); {N<=100}
       for i:=1 to N do read(A[i]);
       read(X);
       A[N+1]:=X; {установка барьера дополнительным элементом}
        i:=1; {i:=0;}
       while A[i]<>X do i:=i+1;
        {repeat i:=i+1; until A[i]=X;}
if i<=N then write('первое вхождение числа ',X,' в массив A на ',i,' месте')
         else write('не нашли');
end.
```

```
program Poisk_Bar2;
         var      A:array[1..100] of integer;
              N,X,i,y:integer;
begin
         read(N);                    {N<=100}
         for i:=1 to N do read(A[i]);
         read(X);
    y:=A[N];              {сохранение последнего элемента}
    A[N]:=X;              {установка барьера на последнее место массива}
     i:=1;                 {i:=0;}
     while A[i]<>X do i:=i+1;
    {repeat i:=i+1; until A[i]=X;}
       if (i<N) or (y=X) then
        write('первое вхождение числа ',X,' в массив A на ',i,' месте')
                        else write('не нашли');
   A[N]:=y;             {восстановление последнего элемента массива}
end.
```

1.3 ДВОИЧНЫЙ (БИНАРНЫЙ) ПОИСК

Алгоритм двоичного поиска можно использовать для поиска элемента только в упорядоченном массиве $a[1] \le a[2] \le \ldots \le a[n]$. Упорядоченность массива позволяет при поиске использовать условия в виде неравенств.

Принцип двоичного поиска:

- Исходный массив делится пополам и для сравнения выбирается средний элемент. Если он совпадает с искомым, то поиск заканчивается.
- Если средний элемент меньше искомого, то все элементы левее его также будут меньше искомого. Следовательно, их можно исключить из процесса дальнейшего поиска, оставив только правую часть массива. Аналогично, если средний элемент больше искомого, то отбрасывается правая часть, а остаётся левая.
- На втором этапе выполняются аналогичные действия над оставшейся половиной массива. В результате после второго этапа останется ¼ массива.
- Процесс продолжается до тех пор, пока или элемент будет найден, или длина зоны поиска станет равной нулю. В последнем случае поиск будет безуспешен.

Алгоритм двоичного поиска имеет вид:

$L = 1$; $R = n$;

пока $(L < R)$ **выполнить**

$m = (L + R)\ div\ 2$;

если $(x > a[m])$ **то** $L = m + 1$

иначе $R = m$

всё-если

всё-пока

Существуют две модификации этого алгоритма для поиска первого и последнего вхождения. Все зависит от того, как выбирается средний элемент: округлением в меньшую или большую сторону. Например, округление c:=(L+R+1) **div** 2 даёт середину с округлением в большую сторону. В случае округления в меньшую сторону средний элемент относится к левой части массива, а при округлении в большую сторону – к правой.

В процессе работы алгоритма двоичного поиска размер фрагмента, где этот поиск должен продолжаться, каждый раз уменьшается примерно в два раза. Это обеспечивает вычислительную сложность алгоритма порядка $\log_2 N$, где N - количество элементов массива.

***ПРИМЕР**: Поиск в упорядоченном по возрастанию массиве первого вхождения числа X.*

```
program Poisk3a;
        var      A:array[1..100] of integer;
            N,X,L,R:integer;
begin
      read(N); {N<=100}
      write('введите упорядоченный по возрастанию массив');
      for i:=1 to N do read(A[i]);
      read(X);
      L:=1; R:=N;    {левая и правая граница фрагмента для поиска}
      while L<R do
     begin
    c:=(L + R) div 2; {середина с округлением в меньшую сторону}
    if X>A[c] then
{если массив упорядочен по убыванию, то if X<A[c]}
L:=c+1            {выбор правой половины без середины с помощью изменения L}
                  else R:=c;
                        {выбор левой половины с серединой с помощью   изменения   R}
end;
if X=A[L] then        {здесь L = R, но не всегда = c}
write('первое вхождение числа ' , X, ' в массив A на позиции ', L)
                  else write('не нашли');
end.
```

1.4. ЗАДАНИЕ

1. Сравнить время выполнения линейного поиска в массиве случайных чисел $a(n)$ при n = 100, 500, 1000, 3000, 10000. В каждом случае использовать не менее трёх массивов одного размера с заданным числом элементов. Построить диаграмму зависимости времени поиска и числа сравнений от числа элементов в массиве.

2. Провести сравнение времени поиска и количества повторений цикла при линейном и двоичном поиске в предварительно отсортированном массиве.

3. Исследовать эффективность введения «барьера» при линейном поиске. Представить таблицы, иллюстрирующие сделанные выводы.

4. В отчёте привести тексты программ с необходимыми комментариями, позволяющими пояснить суть выполняемых действий.

ЛИТЕРАТУРА

1. Кнут Д. Искусство программирования для ЭВМ в 3 т.- М.: Мир, 1978.- Т.3. Сортировка и поиск.
2. Вирт Н. Алгоритмы и структуры данных. – С.-Пб.: Невский диалект.

6. Лабораторная работа. Способы физического представления стека

СТЕК — динамическая структура данных, организованная по принципу «Последним пришёл — первым ушёл». Другое название стека —структура LIFO (это название происходит от англоязычного аналога указанного принципа: «Last In — First Out»). Иногда может применяться раннее название МАГАЗИН, происхождение которого связано с аналогичной организацией оружейного магазина.

Особенностями СТЕКА являются:

a) последовательный доступ к элементам, организованным в СТЕК;
b) неразрушающая запись (при добавлении новых данных в структуру);
c) единственная точка доступа к данным, образующим СТЕК.

Последний элемент, введённый в СТЕК и ещё не удалённый, называется *вершиной* СТЕКА, остальные элементы образуют *тело* СТЕКА. Тело , в свою очередь, является СТЕКОМ.

Допустимыми операциями при работе со СТЕКОМ в классическом варианте считаются следующие:

a) создание СТЕКА:
b) проверка, является ли СТЕК пустым;
c) добавление нового элемента в СТЕК;
d) удаление элемента, находящегося в точке доступа;
e) просмотр данного, находящегося в точке доступа.

При практической реализации стека к этим операциям обычно добавляют возможность просмотра содержимого всей структуры данных и возможность очистки памяти, выделенной под стек, когда потребность в использовании стека отпала.

Стек используется, например, при обработке последовательности активных подпрограмм или вложенных циклов. В этом случае подпрограмма, вызванная последней, закончит свою работу первой. Вложенный цикл завершится ранее охватывающего цикла. Стек образуют омонимические переменные в программе. Стек применяется при работе рекурсивных функций, а также при обработке прерываний.

1.1. Цепное представление стека

Под цепным представлением понимают представление в динамически распределяемой области памяти (в «куче») в виде цепочки элементов, связанных с помощью указателей. В качестве точки входа в цепочку применяют указатель на вершину. Описание типа соответствуюшего указателя на Турбо Паскале может быть дано в форме:

```
Type UKAZ=^STK;
     STK=record
       info: integer;
       next: UKAZ     {указатель на предыдущий элемент стека – следующий элемент цепочки}
           end;
 Var Top, NE, Kon: UKAZ; value: integer;
```

Здесь указатель включён в структуру данных. Поле записи info имеет тип integer. Это означает, что в рассматриваемом примере тип данных, образующих СТЕК, предполагается целочисленым. В случае другого типа данных поле info следует изменить.

Далее рассматривается вариант реализации операций со стеком при цепном представлении. При этом использованы процедуры и функции(реализация на языке Турбо Паскаль).

1. Создание стека (Top- указатель на вершину)

```
Procedure Sozd_St;
    BEGIN
      Top:=Nil;
    END;
```

Замечание. **На практике часто операцию создания стека совмещают с внесением в него начальных данных.**

2. Добавление элемента в стек (Добавляемый элемент – value)

```
Procedure Dobav_St;
        BEGIN
          New(NE);
          NE^.next:=Top;
          NE^.info:= value;
          Top:= NE
        END;
```

3. Удаление элемента из стека

```
Procedure Udal_St;
        BEGIN
          Kon:= Top;      {Для сохранения доступа к фрагменту памяти, занимаемому удаляемым элементом. }
```

```
        Top:=Top^.next;
        Dispose(Kon)  {Возврат памяти в «кучу»}
      END;
```

4.Чтение вершины (последний элемент в стеке)

```
Function Last (S:UKAZ; Var ERR: boolean): integer;
BEGIN
   If S=Nil then begin
                    Writeln('стек пуст'); err:=true;
                     Exit
                   end
    else begin                    {чтение успешно}
            write('вершина=', S^.info:4);
             err:=false;
             Last:= S^.info
         end
END;
```

5. Просмотр элементов стека

```
Procedure Cont_St;
    {вывод элементов в обратном порядке}
    BEGIN
      Kon:= Top;
      while Kon<>Nil do
         begin
           writeln(Kon^.info);
           Kon :=Kon^.next
         end
     END;
```

1.2. Сплошное представление стека

При сплошном представлении элементы расположены в памяти последовательно один за другим. Обычно такое представление предполагает отображение стека на одномерный массив с базовым типом, соответствующим типу данных, размещаемых в стеке. Кроме описания этого массива требуется указатель на вершину стека. Роль такого указателя играет переменная, имеющая тип индекса массива размещения. Поскольку динамическая структура данных (стек) моделируется в статической (одномерный массив), то следует предусматривать проверку переполнения массива при добавлении данных.

Пусть стек размещается в массиве СТЕК [1..n]:Т, имеющем n элементов. Здесь Т – базовый тип элементов массива. Вершина стека соответствует целочисленной переменной ИНДЕКС. Тогда программа создания стека состоит в инициализации переменной ИНДЕКС нулём.

```
1. Программа  СОЗД_СТЕКА;
     начало
          ИНДЕКС:= 0;
     конец;
```

Другие операции можно реализовать следующим образом.

```
2. Программа ДОБАВЛЕНИЕ (аргумент t:T; модифицируемые параметры СТЕК [1..n ]:T; ИНДЕКС: ЦЕЛОЕ);
        начало
           если  ИНДЕКС=n  то ОШИБКА
              иначе
                 начало ИНДЕКС:= ИНДЕКС+1;
                          СТЕК [ИНДЕКС]:= t
                 конец
         конец;

3. Программа УДАЛЕНИЕ (аргумент t:T; модифицируемые параметры СТЕК [1..n ]:T; ИНДЕКС: ЦЕЛОЕ);
      начало
          если ИНДЕКС = 0 то ОШИБКА
             иначе ИНДЕКС = ИНДЕКС - 1
      конец;
```

При сплошном представлении легче адресация, но требуется заранее указывать размер массива, следовательно, либо не используется часть массива (резерв), либо может возникнуть переполнение, если размер массива мал.

ЗАДАНИЕ

1. Разработать подпрограммы операций функциональной спецификации стека при сплошном и цепном представлении (включая проверку на пустоту и просмотр содержимого стека).

2. Разработать демонстрационную программу работы со стеком с возможностью выбора формы физического представления и просмотра результатов операций.

7. Лабораторная работа. Работа с различными представлениями очереди

Тема: ИЗУЧЕНИЕ СПОСОБОВ ФИЗИЧЕСКОГО ПРЕДСТАВЛЕНИЯ ОЧЕРЕДИ.

ОЧЕРЕДЬ — динамическая структура данных, организованная по принципу «Первым пришёл — первым ушёл». Другое название очереди — структура FIFO (это название происходит от англоязычного аналога указанного принципа: «First In — First Out»).

Очередь характеризуется:

- **последовательным доступом** к элементам, организованным в ОЧЕРЕДЬ;
- **неразрушающей записью** (при добавлении новых данных в структуру);
- **двумя точками доступа** к данным, образующим ОЧЕРЕДЬ (одна точка доступа, называемая "хвостом", используется для пополнения ОЧЕРЕДИ, другая, называемая "головой", служит для удаления элемента из ОЧЕРЕДИ).

Допустимыми операциями при работе с ОЧЕРЕДЬЮ в классическом варианте считаются следующие:

f) создание ОЧЕРЕДИ:
g) проверка, является ли ОЧЕРЕДЬ пустой;
h) добавление нового элемента в ОЧЕРЕДЬ;
i) удаление элемента, находящегося в "голове";
j) просмотр элемента, находящегося в "голове".

При практической реализации очереди к этим операциям обычно добавляют возможность просмотра содержимого всей структуры данных и возможность очистки памяти, выделенной под очередь, когда потребность в её использовании в программе отпала. Особенностью операции удаления является то, что не требуется указывать удаляемый элемент, так как всегда удаляется элемент, находящийся в "голове" очереди.

Структуры типа "очередь" используют:

- при рассмотрении задач массового обслуживания (телефонная станция, станция "скорой помощи" и т.п.);
- **при решении проблем функционирования операционной системы компьютера по обслуживанию многозадачного и многопользовательского режимов.**

1.1. Физическое представление очереди

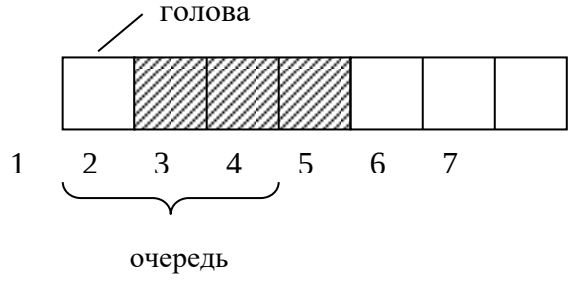
Принято рассматривать два основных способа физической реализации структуры данных ОЧЕРЕДЬ:

1. **Сплошное представление** – соответствующее отображению элементов ОЧЕРЕДИ на одномерный массив.
2. **Цепное представление** – в динамической области памяти, с использованием указателей.

Оба варианта требуют наличия двух точек входа (голова и хвост).

При сплошном представлении особого внимания требует ситуация переполнения массива при добавлении элементов. Пустая очередь при любом физическом представлении приводит к ошибке при попытке бесконтрольного применения к ней операции удаления.

При реализации сплошного представления очереди она отображается на одномерный массив, и вводятся две переменные, имеющие тип индекса массива, которые исполняют роль указателей на "голову" и "хвост".



При сплошном представлении легче адресация, но требуется заранее указывать размер массива, следовательно, либо не используется часть массива (резерв), либо может возникнуть переполнение, если размер массива мал.

Замечание. При представлении очереди удобно предполагать, что указатель "хвост" установлен на первый свободный элемент массива, куда можно добавить элемент очереди. В этом случае облегчается проверка переполнения массива.

1.1.1. Варианты работы со сплошным представлением очереди

1. После удаления элемента из головы все элементы очереди сдвигаются к началу массива. В качестве иллюстрации можно рассмотреть перемещение магнитной ленты над неподвижной считывающей головкой. Недостаток: необходимость перемещения большого числа элементов.
2. Сдвиг производится, если хвост достиг границы массива, то есть нет места для размещения новых данных в очереди, хотя в начале массива есть свободное место. Можно считать, что магнитная лента не перемещается, а передвигается считывающая магнитная головка.

3. Закольцевать массив – вообще отсутствует операция по перезаписи очереди внутри массива размещения. В этом случае перемещаются лишь считывающая и записывающая головки.

1.1.2. Сплошное представление очереди

Пусть T — тип данных, помещаемых в очередь (базовый тип элементов очереди). Тогда для реализации сплошного представления очереди требуется выполнить следующие описания:

1. Массив ОЧЕРЕДЬ [1:n]:T;
2. Переменные хвост (j), голова (i): ЦЕЛЫЕ.

Операции с очередью реализуются в виде подпрограмм, работающих с введенным массивом ОЧЕРЕДЬ, "указателем" i, соответствующим "голове", и "указателем" j, соответствующим "хвосту":

1. При добавлении элемента в хвост очереди переменная j изменяется на 1 (+1 до достижения n, а затем производится переход на 1 элемент массива);
2. При удалении элемента из головы i изменяется на 1 (+1 до достижения n, а затем переход на 1).
3. Очередь пуста — соответствует i = j после выполнения операции удаления.
4. Массив полон — соответствует i = j после выполнения операции пополнения очереди.

Если ввести дополнительный элемент массива с индексом 0, то в нём можно хранить число элементов очереди. Тогда упрощаются проверки переполнения массива и случая пустой очереди. При этом алгоритмы операций с очередью можно представить в виде:

1.
```
Программа Созд_очереди (арг массив ОЧЕРЕДЬ[0:n] : T; голова, хвост : ЦЕЛЫЕ);
  начало
      голова := 1 ;
       хвост := 1 ;
  ОЧЕРЕДЬ[0] := 0
  конец;
```

2.
```
Программа Добавить_элемент (арг : элем :T; массив ОЧЕРЕДЬ[0:n] : T; голова, хвост :ЦЕЛЫЕ);
начало
        если ОЧЕРЕДЬ[0] = n то ОШИБКА
    иначе   начало  ОЧЕРЕДЬ [хвост] := элем;
                    хвост:=хвост+1;
                        ОЧЕРЕДЬ[0] := ОЧЕРЕДЬ[0]+1;
            конец;
  если хвост = n+1 то хвост := 1;
конец;
```

3.
```
Программа Удалить (арг массив ОЧЕРЕДЬ[0:n] : T; голова, хвост : ЦЕЛЫЕ);
начало
        если ОЧЕРЕДЬ[0] = 0 то ОШИБКА
            иначе начало голова := голова+1;
                    ОЧЕРЕДЬ[0] := ОЧЕРЕДЬ[0]-1;
          конец;
        если голова = n+1 то голова := 1;
  конец;
```

4.
```
Программа Первый (арг массив ОЧЕРЕДЬ[0..n]:T;  голова, хвост : ЦЕЛЫЕ)
    начало
      если ОЧЕРЕДЬ[0] = 0 то ОШИБКА
          иначе ПЕРВЫЙ:=ОЧЕРЕДЬ [голова]
      конец ;
```

Замечание 1. ОШИБКА соответствует описанию действий, которые должны быть выполнены.

Замечание 2. В рассмотренных алгоритмах по умолчанию предполагается для простоты, что тип T соответствует типу ЦЕЛЫХ. В других случаях служебная информация о фактической длине очереди может храниться отдельно от массива размещения данных.

1.1.3. Цепное представление очереди

Под цепным представлением понимают представление в динамически распределяемой области памяти (в «куче») в виде цепочки элементов, связанных с помощью указателей. В качестве точки входа в цепочку применяют указатель на голову. Описание типа соответствующего указателя на Турбо Паскале может быть дано в форме:

```
Type UKAZ=^OCHER;
     OCHER=record
       info: integer;
```

```
        next: UKAZ     {указатель на следующий элемент  цепочки}
            end;
    Var H, E, NE, KON: UKAZ; value: integer;
```

Здесь указатель включён в структуру данных. Поле записи info имеет тип integer. Это означает, что в рассматриваемом примере тип данных, образующих ОЧЕРЕДЬ, предполагается целочисленным. В случае другого типа данных поле info следует изменить.

Далее рассматривается вариант реализации операций с ОЧЕРЕДЬЮ при цепном представлении. При этом использованы процедуры и функция (реализация на языке Турбо Паскаль).

1. Создание очереди (H- указатель на голову, E- указатель на хвост)

```
Procedure Sozd_Och;
    BEGIN
        E := Nil;
     H := E
    END;
```

Замечание. **На практике часто операцию создания очереди совмещают с внесением в неё начальных данных.**

2. Добавление элемента в очередь (Добавляемый элемент – value)

```
    Procedure Dobav_Och;
        BEGIN
          New(NE);
          NE^.next:=nil;
          NE^.info:= value;
         If H=nil then H:=NE                {очередь пуста}
                  else E^.next:=NE;          {в конец очереди}
  E := NE                        {E – указатель на последний}
        END;
```

3. Удаление элемента из очереди

```
    Procedure Udal_Och;
        var KP : Ukaz;
        BEGIN
          if H = Nil then
                    begin
                      writeln('пустая очередь – удалить нельзя');
                      exit
                    end
                  else
                      begin
                          KP  :=  H;   {Для сохранения доступа к фрагменту памяти, занимаемому
удаляемым элементом. }
                          H := H^.next;
                          dispose(H)  {Возврат памяти в «кучу»}
                    end
        END;
```

4. Чтение первого элемента в очереди

```
  Function First (H:UKAZ; Var ERR: boolean): integer;
  BEGIN
    If H=Nil then begin
                    Writeln('очередь пуста'); err:=true;
                    Exit
                  end
     else begin                    {чтение успешно}
            write('первый=', H^.info:4);
             err:=false;
             First:= H^.info
          end
  END;
```

5. Просмотр элементов очереди

```
  Procedure Cont_Och;
      BEGIN
        Kon:= H;
        while Kon<>Nil do
          begin
```

```
        writeln (Kon^.info);
        Kon :=Kon^.next
      end
  END;
```

ЗАДАНИЕ

1. Разработать подпрограммы операций функциональной спецификации очереди при сплошном и цепном представлении (включая проверку на пустоту и просмотр содержимого очереди).

2. Разработать процедуру выборки-удаления элемента из очереди, совмещающей взятие первого элемента для обработки с удалением его из очереди.

3. Разработать демонстрационную программу работы с очередью с возможностью выбора формы физического представления (сплошного или цепного) и просмотра результатов операций. При сплошном представлении воспользоваться массивом, проходимым циклически.

4. В отчёт включить текст программы с представлением необходимых комментариев.

8. Лабораторная работа. Организация и алгоритмы обработки линейных списков

Определение. *Линейный список — конечное и упорядоченное множество данных, для которого определены операции добавления и удаления элементов в произвольной точке множества с сохранением упорядоченности.*

Можно сказать, что линейный список представляет собой обобщение структур стеков и очередей. В отличие от элементов этих структур данных любой элемент линейного списка доступен без извлечения других элементов.

Линейный список можно определить как перечисление элементов $l = (l_1, l_2, l_3, l_4, l_5, l_6)$.

По Д. Кнуту понятие «линейный список» отличается от понятия « список», представляющего собой древовидную структуру.

Примеры.

1. Очередь обработки прерываний с учётом различных приоритетов представляет собой линейный список.
2. В трансляторе для синтаксического анализа текста программы используются линейные списки (список идентификаторов в описании переменных; списки параметров переменных; списки полей в записи).

1.1 Функциональная спецификация линейного списка

Считая, что базовый тип элементов Т, можно определить наименование типа «линейный список» как $ЛС_Т$. Тогда перечень допустимых операций будет иметь вид:

1. Создать_список: ( ) $\longrightarrow$ $ЛС_Т$.
2. Список_пуст: $ЛС_Т$ $\longrightarrow$ ЛОГ.
3. Следующий: $Т \times ЛС_Т$ Т. $\longrightarrow$
4. Добавление: $Т \times Т \times ЛС_Т$ $\longrightarrow$ $ЛС_Т$.
5. Удаление: $Т \times ЛС_Т$ $\longrightarrow$ $ЛС_Т$.
6. Первый: $ЛС_Т$ Т. $\longrightarrow$

Из этого перечня следует, что при создании линейного списка он должен быть пустым; проверка линейного списка на пустоту возвращает логическое значение. При добавлении требуется указывать не только добавляемый элемент, но и место включения нового элемента в списке («косой крест» означает операцию декартова произведения). Здесь возможны варианты: «перед каким» или «после какого» требуется вставить элемент. При удалении может быть указан удаляемый элемент или место удаления в списке.

Перечень операций функциональной спецификации следует дополнить перечислением свойств введённых операций. Свойства представляют собой проверки, которые должны выполняться при физической реализации. Количество таких проверок может быть различным. Оно определяется классом задач, где предполагается использование структуры данных.

1.2 Физическая реализация линейного списка

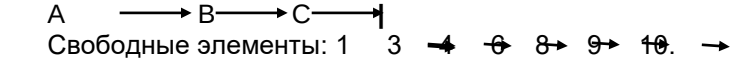
Обычно рассматривают следующие варианты физического представления:

1. **Сплошное представление без использования указателей.** Здесь имеется в виду отображение линейного списка на одномерный массив. Очевидным недостатком такого представления является необходимость перемещения части элементов списка при выполнении операций добавления и удаления.
2. **Сплошное представление с использованием указателей**. При таком представлении не требуется производить перенос элементов списка внутри массива размещения. За такое удобство требуется выделить память под дополнительный массив, где размещаются «указатели», в роли которых выступают индексы массива данных.
3. **Классическое цепное представление с использованием указателей**. В этом случае элементы списка соединяются в цепочку в области динамического размещения данных (в «куче») с помощью указателей, включаемых в элементы цепочки. При этом усложняется процесс программирования, требуется память под указатели, однако не нужно заранее выделять память под массивы размещения.

2.1 Сплошное представление с использованием указателей

Для такого представления надо описать два массива: для размещения данных (Данные) и для размещения указателей (Указатели). Кроме того, необходимы переменные, которые будут играть роль точки входа в структуру (Адр_вх) и указателя на вершину стека свободных элементов (Стек). Если индексация массива начинается с нуля, то признаком окончания цепочки может служить значение нуль в массиве указателей.

Пример.





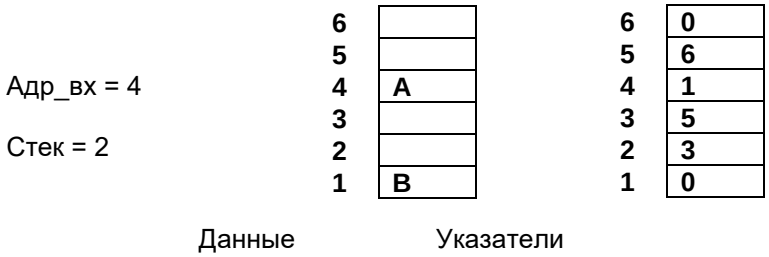
Здесь массивы состоят из 10 элементов.

Стек свободных элементов не требует для своего размещения дополнительной памяти: он помещается в массиве указателей.

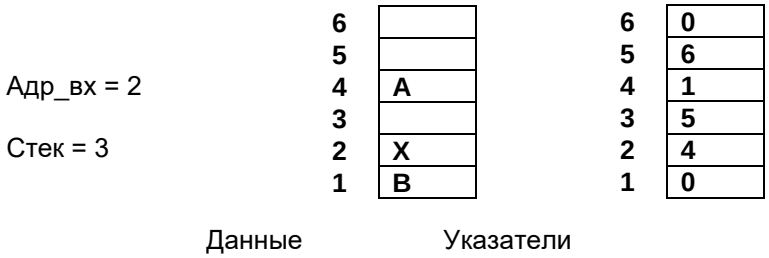
Вершина стека свободных элементов содержит индекс элемента массива данных, где будет помещён добавляемый элемент.

Если добавляемый элемент должен располагаться в начале списка, то начальное и конечное положение в массивах из шести элементов будет выглядеть следующим образом:

Исходное положение



Результирующее положение



Очевидно, были выполнены операции:

Данные [Стек] = X;
t = Указатели [Стек];
Адр_вх = Стек;
Стек = t;

При добавлении в тело списка или в конец списка следует указать «текущий» элемент (устанавливается на элемент, за которым следует добавить новый элемент списка). Тогда фрагмент алгоритма имеет вид:

Данные [Стек] = X;
t = Указатели [Стек];
Указатели [Стек] = Указатели [Текущий];
Указатели [Текущий] = Стек;
Стек = t;

Аналогично записываются алгоритмы удаления элемента из головы списка и из хвоста.

Замечание. Следует проверять, не переполнится ли массив при добавлении элемента. При удалении элемента линейного списка надо убедится, что список не пуст.

2.2 Цепное представление с использованием указателей

Под цепным представлением понимают представление в динамически распределяемой области памяти (в «куче») в виде цепочки элементов, связанных с помощью указателей. В качестве точки входа в цепочку применяют указатель на голову. Описание типа соответствующего указателя на Турбо Паскале может быть дано в форме:

```
Type UKAZ=^ES;
        ES=record
          info: integer;
          next: UKAZ     {указатель на следующий элемент  цепочки}
               end;
 Var r, q, H, NE: UKAZ; value: integer;]
{H – первый элемент, q – текущий}
```

Здесь указатель включён в структуру данных. Поле записи info имеет тип integer. Это означает, что в рассматриваемом примере тип данных, образующих линейный список, предполагается целочисленным. В случае другого типа данных поле info следует изменить.

Далее рассматривается вариант реализации операций с линейным списком при цепном представлении. При этом использованы процедуры и функция (реализация на языке Турбо Паскаль).

1. Создание линейного списка (H- указатель на голову)

```
Procedure Sozd_Sp;
    BEGIN
       H := nil
    END;
```

Замечание. На практике часто операцию создания линейного списка совмещают с внесением в него начальных данных.

2. Добавление элемента (Добавляемый элемент – value, вставляется после элемента, на который указывает q).

```
Procedure Dobav_Sp;
        BEGIN
           New(NE);
           NE^.info:= value;
           NE^.next:=q^.next;
           q^next:=NE;
         END;
```

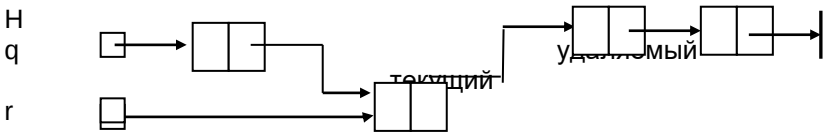
Замечание. Не предусмотрена возможность добавления перед началом списка.

3. Удаление элемента, следующего за тем элементом, на который указывает q

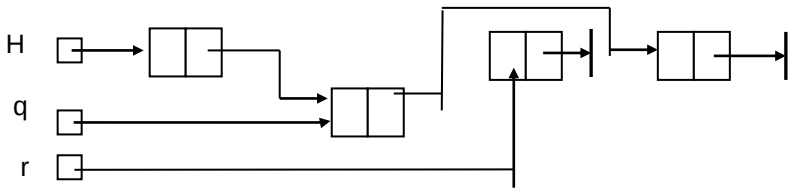
```
Procedure Udal_Sp;
    BEGIN
      r := q^.next;  {для сохранения доступа к фрагменту памяти, занимаемому удаляемым элементом}
      q^.next := q^.next.^next;     {удаление из цепочки}
      r^.next:=nil;                   {завершение обхода}
       dispose(r)                  {возврат памяти в «кучу»}
      END;
```

Операцию удаления можно наглядно представить в форме:

A). Исходное состояние



B). Результирующее состояние



Замечание. Не предусмотрена возможность удаления первого элемента списка (у него нет предшествующего).

4. Добавление value в начало линейного списка

```
procedure Dob_HSp;
   BEGIN
```

```
    new(NE);
    NE^.info:= value;
    NE^.next:= H;
    H:= NE
  END;
```

ЗАДАНИЕ

1). Разработать программу, демонстрирующую работу с линейным списком при сплошном и цепном представлении.

2). При сплошном представлении без использования указателей спроектировать счётчики перемещаемых элементов списка в массиве.

2). Предусмотреть реализацию операций функциональной спецификации в виде подпрограмм.

3). Добавить подпрограммы вывода всего списка, а также полного удаления списка.

4). Предусмотреть меню выбора варианта представления линейного списка, а также выбора операций с возможностью продолжения работы.

5). Представить письменный отчёт о работе, приводя тексты программ с необходимыми комментариями.

Литература

1. ***Вирт Н. Алгоритмы и структуры данных. С-Пб.: Невский диалект, 2008***

2. Кнут Д. Искусство программирования, т.3. М.: Вильямс, 2010.

3. Ахо А., Хопкрофт Дж., Ульман Дж. Структуры данных и алгоритмы. М.: Вильямс, 2010.

9. Лабораторная работа. Основные операции с деревьями

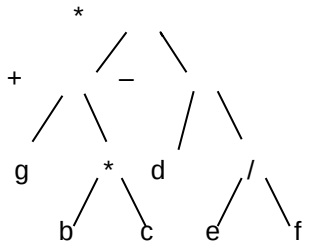
Определение. Двоичным (бинарным) деревом называется упорядоченное дерево степени 2 (*лпд, ппд* — левое и правое поддерево).
Определение. Деревья со степенью больше двух называются сильноветвящимися.

Алгоритмы обхода двоичного дерева

Алгоритм обхода дерева — алгоритм, в котором доступ к каждой вершине дерева осуществляется один и только один раз.
Обход двоичного дерева сводится к выполнению трёх действий:
1) обработка корня (К);
2) обход левого поддерева (Л);
3) обход правого поддерева (П).

В зависимости от очерёдности этих действий различают следующие способы обхода:
1) «сверху вниз» (префиксный) = КЛП;
2) «снизу вверх» (постфиксный) = ЛПК;
3) «слева направо» (инфиксный) = ЛКП.

Замечание 1. Рассмотренный порядок обхода является левым. Можно ввести правый порядок обхода поддеревьев (КПЛ, ПЛК, ПКЛ).
Замечание 2. Алгоритм обхода не зависит от вида обработки вершин. Он является общим для алгоритмов, выполняющих различную обработку.
Пример.



1) КЛП: *(+(g,*(b, c)), –(d, /(e, f))) (префиксный обход: запятая — разделитель поддеревьев);
2) ЛПК: ((g, (b,c)*)+, (d, (e,f)/)–)* — постфиксный обход;
3) ЛКП: (g+(b*c))*(d– (e/f)) — инфиксный обход; корень — разделитель поддеревьев).

Алгоритм печати дерева

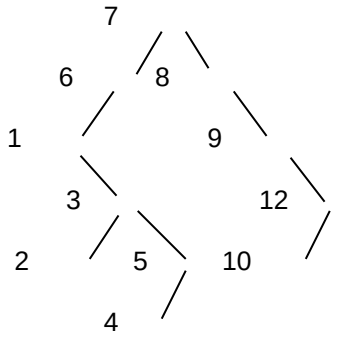
Пусть a — двоичное дерево. Рекурсивный алгоритм печати дерева можно представить в форме:

```
печать_дерева(a);
начало
если not(дерево_пусто (a)) то
 начало
     печать_корня (чтение_корня (a));    {печать корня}
         печать_дерева (левое_поддерево (a));      {рекурсия}
         печать_дерева (правое_поддерево (a));
     конец;
     конец;
```

Замечание. При разработке алгоритма печати дерева использован алгоритм обхода дерева «сверху вниз» (КЛП).

Двоичное дерево поиска

Пусть для элементов типа T установлено отношение порядка. Если множество таких элементов организовать в структуру, являющуюся двоичным деревом, у которого в левом поддереве содержатся элементы с ключом, меньшим, чем ключ узла, а в правом — с ключом, большим, чем ключ узла, то такое дерево называется двоичным деревом поиска.
Пример. Пусть данные поступают из файла в следующем порядке: 7,6,8,1,3,5,4,2,9,12,10. Двоичное дерево поиска из этих элементов будет иметь вид:



Включение элемента в двоичное дерево поиска

Алгоритм включения элемента в двоичное дерево поиска можно представить следующим образом.

```
программа включение: ДДТ (аргументы t:T; a : ДДТ);
    начало
    включение ◄──если дерево_пусто( a ) то
          построение (t, ПУСТО, ПУСТО)
       иначе
         начало если t< чтение_корня( a )
                 то
                      построение ( чтение_корня ( a ),
                                 включение(t, лпд( a )), ппд( a ))
                 иначе
                      построение (корень( a ),
                                 лпд( a ), включение( t, ппд( a )))
        конец
    конец
```

Замечание. Элемент t проходит по дереву, начиная от корня, поворачивая вправо или влево от каждой вершины в зависимости от значения, пока не попадает на пустое место. На это место он и записывается.

Программа построения двоичного дерева поиска из элементов 3, 1, 4, 2, 5, 8 имеет вид:

```
    начало
 a ←      создание_дерева;
        включение(3, a );
        включение(1, a );
        включение(4, a );
        включение(2, a );
        включение(5, a );
        включение(8, a );
 конец
```

Удаление из двоичного дерева поиска

Пусть задано двоичное дерево поиска. Надо удалить из него вершину с указанным значением ключа, не нарушая упорядоченности. Возможные варианты:
1). Удаляемая вершина — лист. Удаление происходит без перестройки дерева (соответствующий указатель получает значение ПУСТО).
2). Удаляемый элемент имеет одного потомка. В цепочке указателей производится обход удалённого элемента.
3). Удаляемый элемент имеет двух потомков. В этом случае возможны два варианта перестройки дерева. При этом удаляемую вершину заменяют либо на самый правый элемент её левого поддерева (самый большой в поддереве), либо на самый левый элемент её правого поддерева (самый маленький в поддереве).

Физическое представление двоичного дерева

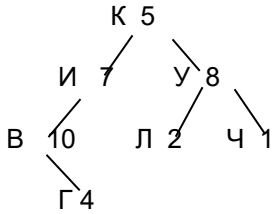
Используются следующие основные способы представления двоичного дерева:
1) Цепное с использованием указателей.
2) Сплошное с использованием указателей.
3) Сплошное без использования указателей.

а). Для цепного представления каждый элемент должен содержать два указателя на левое и правое поддерево:

```
Type ptree= ^elem;
     elem= record
          info: <тип данных>;
          left, right: ptree
             end;
```

б). Сплошное представление с использованием указателей требует описать массив данных для записи ключей и связанных с этим массивом двух массивов указателей на левые и правые поддеревья. Необходимы также две переменные АДРЕС_ВХОДА и СТЕК, которые будут указывать на начало структуры данных и вершину стека свободных элементов.
Этот стек можно представить в одном из массивов указателей на поддеревья. Если нумерация элементов массивов начинается с 1, то удобно считать, что указатель=0 соответствует структуре данных ПУСТО.

Пример.



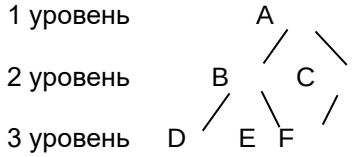
Представленное дерево является двоичным деревом поиска (порядок алфавитный). Рядом с узлами дерева указаны номера элементов массива данных, в которых находятся соответствующие ключи.

	Данные		ЛПД		ППД
10	В	10	0	10	4
9		9		9	0
8	У	8	2	8	1
7	И	7	10	7	0
6		6		6	9
5	К	5	7	5	8
4	Г	4	0	4	0
3		3		3	6
2	Л	2	0	2	0
1	Ч	1	0	1	0

Здесь АДРЕС_ВХОДА = 5; СТЕК = 3.

в). Сплошное представление без использования указателей (в одном массиве) удобно для так называемых полных деревьев, то есть таких, у которых все вершины заняты, кроме, может быть, нескольких «правых» в ряду последнего уровня. Для отображения на массив последовательно записываются элементы каждого уровня слева направо.

Пример.



Замечание. Здесь уровни для удобства считаются с 1, а не с 0, как в классическом варианте.

На одномерный массив это дерево отображается следующим образом:

1	2	3	4	5	6	7	8
A	B	C	D	E	F		

а). Потомки элемента с номером i имеют номера $2i$ и $2i+1$.

б). Элемент j уровня i имеет индекс $\left[2^{i-1}+j-1\right]$.

Замечание. В рассмотренном представлении неявно использован алгоритм обхода «по уровням», отличающийся от рассмотренных ранее способов обхода.

Поиск ключа в двоичном дереве поиска

Алгоритм поиска, возвращающий поддерево, корнем которого является ключ поиска x, можно представить в форме:

```
программа ПОИСК :ДДТ (арг x:T;  a : ДДТ);
    найден ← false;
пока  not (дерево_пусто( a )) and not (найден)
   начало
     если  x = чтение_корня( a ) то найден=true
          иначе
             начало
              если  x  <  чтение_корня( a )
                 то   a ← ПОИСК( x , лпд( a ))
                 иначе  a ← ПОИСК( x , ппд( a ))
              конец;
   конец;
ПОИСК ← a ;
```

ПОИСК С БАРЬЕРОМ В ДВОИЧНОМ ДЕРЕВЕ ПОИСКА

Если установить барьер, то есть добавить элемент, информационное поле которого совпадает с ключом поиска, то можно упростить условие окончания цикла поиска (не надо проверять, что очередное поддерево не пусто).

Барьер можно рассматривать как общее представление всех внешних вершин. Иногда барьер называют якорем, а структуру с барьером — «гамаком».

Очевидно, что при наличии барьера ключ поиска всегда будет найден. Для определения результата поиска следует после окончания цикла поиска проверить, с каким указателем найден ключ. Если найден барьер, то результат поиска отрицательный.

ЗАДАНИЕ

1). Разработать подпрограмму построения двоичного дерева поиска с указанными элементами. Предусмотреть возможность ввода данных с клавиатуры или из предварительно приготовленного файла.
2). Разработать подпрограмму поиска элемента в двоичном дереве поиска без барьера и с возможностью установки барьера. Установить счётчики числа сравнений в процессе поиска.
3). Разработать подпрограмму добавления элемента в двоичное дерево поиска.
4). Разработать подпрограмму удаления элемента из двоичного дерева поиска.

Литература

1. *Вирт Н. Алгоритмы + структуры данных = программы. М.: Мир, 1985*
2. Кнут Д. Искусство программирования, т.3. М.: Вильямс, 2000.
3. Ахо А., Хопкрофт Дж., Ульман Дж. Структуры данных и алгоритмы. М.: Вильямс, 2000.

10. Лабораторная работа. Разрешение конфликтов при хешировании

Хеширование — метод размещения и поиска данных. Каждому элементу множества данных (ключу) ставится в соответствие целое число, которое называется хеш-кодом или хеш-значением. Термин *хеширование* происходит от английского слова *hash* — перемешивание, перемалывание.
В печатных работах по программированию этот термин появился в 1967 г., хотя сам механизм был известен и ранее.
Целью хеширования является быстрый поиск данных. Задача хеширования может быть сформулирована следующим образом: найти подходящее отображение H ключей K в адреса A в памяти компьютера ($H : K \to A$).
Поскольку доступ к элементам массива прямой, то в качестве модели памяти можно использовать одномерный массив. Тогда можно считать, что данные отображаются на массив. При этом H – отображение, преобразующее ключи в индексы массива. Последнее привело к другому названию метода («преобразование ключей» или «расстановка ключей»).
Основная трудность реализации хеширования состоит в том, что множество возможных значений существенно шире множества допустимых адресов (индексов массива).

1. ВЫБОР ФУНКЦИИ ХЕШИРОВАНИЯ

В идеальном варианте для задач поиска хеш-адрес должен быть уникальным, чтобы за одно обращение получить доступ к элементу, характеризуемому заданным ключом. На практике приходится идти на компромисс и исходить из того, что множества данных с одинаковым хеш-адресом могут содержать более одного элемента. Это приводит к тому, что поиск ключа может потребовать более одного шага.
Определение. Ситуация в методе хеширования, когда «в указанном месте» ключа нет, называется **конфликтом** или **коллизией**. Задача формирования альтернативного индекса называется разрешением конфликта.
Основные требования к функции преобразования ключей:
1. Равномерное распределение ключей по всему диапазону значений индекса.
2. Эффективное вычисление (функция должна содержать небольшое число операций).

Часто функция хеширования представляется в форме:

$$H(k) = \phi(k) \bmod N .$$

Здесь N – размер массива; $\phi(k)$ – первичная функция преобразования ключа. Операция деления по модулю (mod) — остаток от деления нацело. Она необходима, чтобы вычисленный индекс попал в допустимый диапазон. Так как остаток может быть нулевым, то массив размером N следует рассматривать с индексом в диапазоне $0..N-1$.

Если $ord(k)$ – функция преобразования ключа k в его порядковый номер во множестве возможных значений, то в качестве $H(k)$ можно взять функцию

$$H(k) = ord(k) \bmod N .$$

Для этой функции характерно равномерное отображение значений ключа на весь диапазон изменения индекса [1]. Кроме того, при N, равном степени 2, эта функция эффективно вычисляется.

Замечание 1. Часто используется функция расстановки на основе «исключающего или», применяемого к некоторой части ключа, рассматриваемого как последовательность двоичных цифр.

Замечание 2. При выборе функции $\phi(k)$ можно считать, что ключ образован из смежных «полей» , например, для строк — из символов, составляющих строку. Тогда ключу можно поставить в соответствие числовой код, образованный из значений этих полей, например, из номеров букв в алфавите.
В качестве примера применим функцию «сумма числовых кодов букв» к множеству ключей БЕГЕМОТ, ВОЛК, ЖИРАФ, ЗЕБРА, КОБРА, ПИТОН при N =10.

1) $H('БЕГЕМОТ')$ = (2+6+4+6+14+16+20) mod 10 = 8;
2) $H('ВОЛК')$ = (3+16+13+12) mod 10 = 4;
3) $H('ЖИРАФ')$ = (8+10+18+1+22) mod 10 = 9;
4) $H('ЗЕБРА')$ = (9+6+2+18+1) mod 10 = 6;
5) $H('КОБРА')$ = (12+16+2+18+1) mod 10 = 9;
6) $H('ПИТОН')$ = (17+10+20+16+15) mod 10 = 8.

Видно, что возникло две коллизии: КОБРА – ЖИРАФ; ПИТОН – БЕГЕМОТ.

2. ВНЕШНЕЕ РАЗРЕШЕНИЕ КОЛЛИЗИЙ

Традиционно принято выделять две схемы хеширования:
- хеширование с цепочками (со списками);
- хеширование с открытой адресацией.

Очевидно, что в первом случае процесс заполнения хеш-таблицы будет достаточно простым, но при этом доступ к элементам потребует двух операций: вычисления индекса и поиска в соответствующем списке. Операции по занесению и поиску элементов при таком виде хеширования будут вестись в незамкнутом (**открытом**) пространстве памяти.

Во втором случае *«открытая адресация»* соответствует тому, что адрес размещения заранее не определён, так как заполнение хеш-таблицы и доступ к элементам может быть весьма замысловатым. Хеш-адрес элемента с данным ключом как бы открыт: он постепенно уточняется. При этом все операции ведутся в **замкнутом** пространстве (в одномерном массиве).

Как нетрудно заметить, традиционная классификация методов хеширования может быть заменена другими: хеширование с цепочками можно относить по классификации Ахо, Хопкрофта и Ульмана [3] к так называемому «открытому» хешированию, а хеширование с открытой адресацией — к «закрытому» хешированию (в одном массиве).

При внешнем разрешении конфликтов элементы размещаются вне выделенного массива. При этом предполагается, что по соответствующему адресу (в массиве) размещается не элемент, указатель на список элементов, имеющих одно и то же значение функции расстановки $H(k)$.

При таком способе разрешения конфликтов отделяется процесс предоставления памяти под массив указателей (статическая структура) и процесс управления самими элементами (динамически в зависимости от потребности). Очевидно, что при таком методе число размещаемых элементов n может превосходить размер N массива.

Если для ключей установлено отношение порядка, то удобно использовать упорядоченные линейные списки. Конечно, при размещении данных потребуются дополнительные проверки.

Как отмечает Дональд Кнут [2], предложение использовать для разрешения коллизий хеш-адресов метод цепочек впервые было высказано Г.П. Ланом (IBM) в январе 1953 г.

Возвращаясь к рассмотренному ранее примеру, можно результат разрешения конфликтов представить в следующей форме.



Рис.1. Результат разрешения коллизий.

Замечание 3. При больших объёмах данных можно элементы массива (указатели) связать с файлами на диске.

Замечание 4. Если к указателям в массиве присоединить двоичные деревья поиска или Б-деревья, то эффективность поиска будет усилена за счёт усложнения алгоритма.

ЗАДАНИЕ

1) Организовать хеширование данных, представляющих собой текстовые строки, используя хеш-функцию на основе суммирования номеров букв в алфавите.
2) Разместить данные в линейных списках, точки входа в которые находятся в массиве указателей заданного размера N. Элементы каждого списка представляют собой ключи, имеющие одно и то же хеш-значение.
3) Организовать поиск данных на основе полученной хеш-структуры с определением числа шагов поиска. Сравнить результаты в случае произвольного и упорядоченного размещения ключей в списках.
4) Организовать размещение конфликтующих элементов в файлах на диске. Продемонстрировать процесс поиска элемента в этом случае.
5) Предложить другую функцию хеширования и показать полученные результаты.

Литература

1. *Вирт Н. Алгоритмы + структуры данных = программы. М.: Мир, 1985*
2. Кнут Д. Искусство программирования, т.3. М.: Вильямс, 2000.
3. Ахо А., Хопкрофт Дж., Ульман Дж. Структуры данных и алгоритмы. М.: Вильямс, 2000.